

# Cumhuriyet Üniversitesi İşletim Sistemleri Dersi

## Bölüm 5 *Süreç Senkronizasyonu* *(Process Synchronization)*

Dr. Halil ARSLAN

# Bölümün hedefleri

- Süreç senkronizasyonu
- Kritik bölüm problemi
- Peterson çözümü
- Senkronizasyon donanımı
- Mutex kilitleri
- Semaforlar
- Klasik senkronizasyon problemleri
- Monitörler
- Senkronizasyon örnekleri
- Alternatif yaklaşımlar
- Deadlocks

# Süreç senkronizasyonu

- İşbirliği gerektiren bir süreç sistemde yürütülen diğer süreçleri etkileyebilir.
- İşbirliği gerektiren süreçler ya aynı hafıza alanını kullanır yada sadece dosya yada mesaj yoluyla verilerini paylaşabilir.
- Paylaşılan verilere eşzamanlı erişim veri tutarsızlıklarına neden olabilir.
- **Süreç senkronizasyonu** bir mantıksal adres alanını paylaşan süreçlerin, işbirliği durumunda **veri tutarlılığını** sağlamak için çeşitli mekanizmalar sunar.
- Süreçler eşzamanlı ve paralel olarak yürütülebilir. Dolayısıyla eşzamanlı yada paralel olarak yürütülen süreçlerin paylaştıkları veri bütünlüğünün **korunması** gerekir.
- Örneğin, bölüm 3'te ele alınan üretici-tüketici problemi, sınırlı tampon kullanımı durumlarında paylaşılan veriye erişim senkronizasyonu olarak modellenenebilir.

# Süreç senkronizasyonu

- Bölüm 3'te üretici/tüketici probleminde tampona en fazla BUFFER\_SIZE-1 eleman atılmasına izin verildi.  $((in+1) \% BUFFER\_SIZE) == out \rightarrow$  tampon dolu  
buffer\_size=10, in=9 (ilk boş alan), out=0 (ilk dolu alan) durumunda 9 boşken tampon dolu olarak algılanır.
- Bu sorunu çözmek için başlangıçta değeri 0 olan `counter` ekleyelim.

```
while (true) { /*üretici*/
    while (counter == BUFFER_SIZE); /*dolu*/
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

```
while (true) { /*tüketici*/
    while (counter == 0); /*boş*/
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
}
```

- Bu rutinler ayrı ayrı çalıştığında doğru olsa da eşzamanlı çalıştıklarında sonuç doğru olmayabilir.
- Örneğin, `counter=5` iken, üretici ve tüketici rutinleri eşzamanlı çalışırken `counter++` ve `counter--` yürütülürse son değer 4, 5 yada 6 olabilir. Diyelimki üretici veriye erişti ve artırdı (6), aynı anda tüketici veriye erişti ve azalttı (4), ikisi birden veriyi yazdı ise sonuç ?. Oysa ayrı ayrı çalıştırılırsalardı son durumda değer 5 olacaktı.

# Süreç senkronizasyonu

- İki işlemin makinede işleyişi; `counter++` ve `counte--` için;

```
register1 = counter
register1 = register1 + 1
counter = register1
```

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- İki süreç sıralı olarak yürütüldüğünde aşağıdaki gibi olabilir;

|            |          |         |                                  |                 |
|------------|----------|---------|----------------------------------|-----------------|
| <b>T0:</b> | üretici  | çalışır | <b>register1 = counter</b>       | {register1 = 5} |
| <b>T1:</b> | üretici  | çalışır | <b>register1 = register1 + 1</b> | {register1 = 6} |
| <b>T2:</b> | tüketici | çalışır | <b>register2 = counter</b>       | {register2 = 5} |
| <b>T3:</b> | tüketici | çalışır | <b>register2 = register2 - 1</b> | {register2 = 4} |
| <b>T4:</b> | üretici  | çalışır | <b>counter = register1</b>       | {counter = 6}   |
| <b>T5:</b> | tüketici | çalışır | <b>counter = register2</b>       | {counter = 4}   |

- Bu durumda tampondaki eleman sayısı 4 olarak görünecektir fakat gerçekte tamponda 5 eleman vardır.
- Aynı veri yapısına, aynı anda birden çok sürecin erişmesi ve işlemesi **yarış durumunu (race condition)** oluşturur.
- Bu durumu engellemek için aynı anda yalnızca bir sürecin erişimine izin vermek yani süreçleri senkronize etmek gerekir. Benzer şekilde bu yapı **multicore** sistemlerde **multithread** uygulamalar için de gereklidir.

# Kritik bölüm problemi

- Bir sistemin ( $P_0, P_1, P_2, \dots, P_{n-1}$ ) tane sürece sahip olduğunu düşünelim,
- Her süreç, ortak değişkenlerin değiştirildiği, tabloların güncellendiği, bir dosyanın yazıldığı gibi kritik bölüm (**critical section**) olarak isimlendirilen bir alana sahip olabilir.
- Burada sistemin önemli özelliği, bir süreç kritik bölümünü yürütürken, diğer süreçlerin onun kritik bölümünü yürütmesine izin vermemesidir.
- Her süreç kendi kritik bölümüne giriş izni veren bir protokol yapısı kullanır.
- Bu yetkilendirme süreçlerde **entry section** bölümünde yapılır. Kritik bölümü **exit section** bölümü takip eder. Diğer kodlar **remainder section** bölümünde yer alır.

```
do {  
    entry section          /*giriş bölümü*/  
        critical section  /*kritik bölüm*/  
    exit section          /*çıkış bölümü*/  
  
        remainder section /*kalan bölüm*/  
} while (true);
```

# Kritik bölüm problemi

- Kritik bölüm probleminin çözümü için aşağıdaki üç gereksinim karşılanmalıdır;
  1. **Mutual exclusion** (karşılıklı dışlama): Eğer bir süreç kritik bölümde çalışıyorsa, diğer hiçbir süreç kritik bölümünü çalıştıramaz.
  2. **Progress** (yürütüm): Hiçbir süreç kritik bölümünü çalıştırmıyorsa ve bazı süreçler kritik bölümüne girmek istiyorsa, remainder section çalıştıranların dışındakilerden birine giriş izni verilir.
  3. **Bounded waiting** (sınırlı bekleme): Hiçbir süreç bir kaynağa erişim için sınırsız olarak bekletilemez. Bir süreç kritik bölüme eriştikten sonra tekrar tekrar diğer süreçlerin erişmesini engelleyecek şekilde yetkilendirilemez. Erişim isteği sırası ile erişim zaman planı kriterlerine göre süreçler önceliklendirilebilir.
- Kritik bölüm problemi işletim sistemlerinin kernel kod'larında da oluşabilir. Örneğin sistemde açık tüm dosyaları tutan bir veri yapısı listesi olsun. İki süreç aynı anda aynı dosyayı açmak istediğinde yine **race conditions** durumu oluşabilir. Benzer durumlar başkaca kernel veri yapıları için de gerçekleşebilir.

# Kritik bölüm problemi

- İşletim sistemlerinde kritik bölümleri işlemek için **preemptive** (kesme öncelikli) **kernels** ve **nonpreemptive kernels** olarak iki temel yaklaşım vardır.
  - **Preemptive:** Bir süreç kernel modda çalışırken, race condition durumları oluşabilir. Yarış durumları dikkatlice programlanmalıdır. Linux 2.6 ve sonrası kernellar, yeni nesil Windows dağıtımları ve bazı UNIX dağıtımları kullanır. Kesme öncelikli çalışır.
  - **Nonpreemptive:** Bu modda çalışan sistemlerde, bir süreç kernel modda iken interrupt'lara izin vermez. Bu nedenle çekirdek veri yapılarında race conditions oluşmaz. Windows 3.1, bazı UNIX dağıtımları ve Linux 2.6 öncesi kernellar.
- Preemptive kernel tasarlanan işletim sistemlerinde çoklu kernel süreçleri desteklediği için kritik bölüm problemi dikkatlice tasarlanmalıdır. Ancak bu kernel'ların cevap verebilirliği daha iyidir ve gerçek zamanlı sistemler için daha uygundur.
- Nonpreemptive kernel tasarlanan işletim sistemlerinde kernel moddaki bir sürecin tamamlanma zamanlaması kestirilemez.



# Peterson Çözümü

- Bu çözüm, kritik bölüm sorununa **Peterson's solution** olarak bilinen yazılım tabanlı çözüm sunmaktadır.
- Bu çözümün, modern bilgisayar mimarilerinde load ve store gibi temel makine dili komutlarının uygulanması yüzünden, **bazı mimarilerde doğru çalışacağının garantisi yoktur.**
- Ancak bu çözüm, kritik bölüm sorununun çözümüne iyi bir algoritma tanımlar ve kritik bölüm sorununun çözümünde **mutual exclusion, progress ve bounded waiting** gereksinimleri için yazılım tasarımlarını göstermektedir.
- Peterson çözümü **critical sections ve remainder sections arasında yürütülen iki süreçle sınırlıdır.**

# Peterson Çözümü

- Bu süreçler  $P_i$  ve  $P_j$  olsun. Peterson çözümü iki veri ögesini paylaşan iki süreç için;

```
int turn;
boolean flag[2];
```

- `turn` değişkeni, kritik bölüm giriş göstergesi olarak kullanılır. Eğer `turn==i`'ise  $P_i$  süreci, kritik bölümünün çalışmasına izin verir.
- `flag` dizisi, bir sürecin kritik bölüm girişine hazır olduğunu gösterir.
- Örneğin `flag[i]==true` ise  $P_i$ 'nin hazır olduğunu gösterir.

```
do { /*Pi sürecinin yapısı*/
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);

    critical section

    flag[i] = false;

    remainder section
} while (true);

/*Pi kritik bölüm için hazır*/
/*Pj hazırsa önceliği ona ver*/
/*Pj hazır ve giriş isteği varsa BEKLE*/

/*Kritik bölümü yürüt*/

/*Kritik bölüm bitti*/

/*Sonraki bölüm*/
```

# Peterson Çözümü

Peterson çözümü kritik bölüm problemi gereksinimlerini karşılar;

- **Mutual exclusion** (karşılıklı dışlama): Bir süreç kritik bölümünde çalışırken diğer sürecin kritik bölümünü çalıştırması `flag[]` ile engellenir. `turn` değişkeni iki süreç tarafından aynı anda değiştirilse bile son değere göre diğer süreç bloklanır.
- **Progress** (yürütüm): Kritik bölümünü tamamlayan süreç kritik bölüm kullanma isteğini `flags` dizisini `false` yaparak iptal eder. Böylece diğer sürecin kritik bölümünü yürütümü sürdürebilmesi sağlanır.
- **Bounded waiting** (sınırlı bekleme): Bir süreç kritik bölüme birkez girdikten sonra sıra diğerine geçer.

```
do { /*Pi sürecinin yapısı*/
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
    critical section
    flag[i] = false;
    remainder section
} while (true);
```

```
/*Pi kritik bölüm için hazır*/
/*Pj hazırsa önceliği ona ver*/
/*Sıra Pi'ye gelen kadar bekle*/
/*Kritik bölümü yürüt*/
/*Kritik bölüm bitti*/
/*Sonraki bölüm*/
```

# Senkronizasyon Donanımı

- Kritik bölüm probleminin çözümü için pekçok yazılım tabanlı çözüm olsa da modern bilgisayar sistemlerinde çözüm garanti edilemez.
- Kritik bölüm probleminin çözümü ister yazılım tabanlı ister donanım tabanlı olsun temel yaklaşım kilitleme (**locking**)'dir.
- Tek işlemcili bir ortamda kritik bölümlerdeki veriler işlenirken **interrupt** (kesme) istekleri engellenerek bu problem önlenabilir. Bu yaklaşım genellikle **nonpreemptive** çekirdek yaklaşımıdır.
- Bu yöntem, kesmelerin devre dışı bırakımında tüm işlemcilere mesaj iletiminin verimsizliği ve daha önemli kesmelerin de engellenebileceği için çok işlemcili yapılar için uygun değildir.
- Bu nedenle çoğu modern sistem, bir alanın içeriğinin güncellenmesi (`test_and_set()` gibi) yada iki alanın içeriğinin yer değiştirilmesi (`compare_and_swap()` gibi) gibi **atomic** çalışan özel donanım komutları sağlar. (Makineye özgü donanım komutları)
- Makineye özgü bu komutlar kullanılarak daha basit bir şekilde kritik bölüm problemi çözülebilir.

# Senkronizasyon Donanımı

- Makineye özgü bu yapıların sanal olarak tanımlandığı `test_and_set()` komutu ve mutual-exclusion (karşılıklı dışlama) entegrasyonu aşağıdaki gibi olabilir.

```
boolean test_and_set(boolean *target){  
    boolean rv = *target;  
    *target = true;  
    return rv;  
}
```

```
do {  
    while (test_and_set(&lock));  
    /* kritik bölüm */  
    lock = false;  
    /* sonraki bölüm */  
} while (true);
```

- Burada `test_and_set()` yordamının **atomic** çalıştığı unutulmamalıdır. Yani bu komut aynı anda iki kez çalıştırılırsa dahi sırayla işlenecektir.
- Fonksiyona geçilen `lock` parametresinin adres uzayındaki değeri kontrol edilir ve  $P_i$  diğer süreçler için kiliti aktif ederken, kendisi değişiklik öncesi değeri kontrol ederek kilitin pasif olması durumunda kritik bölümü çalıştırmaya başlar.
- $P_i$  için kritik bölüm tamamlandığında kilit pasif edilir ve sonraki süreçlerin kritik bölümlerine girmesi sağlanır.

# Senkronizasyon Donanımı

- Benzer şekilde `compare_and_swap()` komutu ve mutual-exclusion (karşılıklı dışlama) entegrasyonu aşağıdaki gibi tanımlanmıştır.

```
int compare_and_swap(int *value,  
    int expected, int new_value)  
{  
    int temp = *value;  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

```
do {  
    while(compare_and_swap(&lock,0,1)!=0);  
    /* kritik bölüm */  
    lock = 0;  
    /* sonraki bölüm */  
} while (true);
```

- Fonksiyonun aldığı `value` parametresi yalnızca `expected` parametresine eşit olduğunda `new_value` ile güncellenir. Ancak fonksiyon daima global değişkenin (`lock`) ilk değerini geriye dönderir.
- Bu yapıda karşılıklı dışlama; başlangıçta 0 olan `lock` değişkeni, 1'e kurulmak istenir (kilidi aktif et). İstenilen güncelleme yapılır ancak  $P_i$  süreci için hale `lock` 0'dır ve kritik bölüme girilir.  $P_i$  kritik bölümünü yürütürken diğer süreçler `lock` değişkenini güncelleyemeyecektir ve `lock` 1 dönecektir.

# Senkronizasyon Donanımı

- Bu komutlar mutual-exclusion (karşılıklı dışlama) gereksinimine çözüm getirir de bounded-waiting (sınırlı bekleme) gereksinimini karşılamamaktadır.
- Bu gereksinim için `test_and_set()` yapısı aşağıda sunulmuştur.
- Kullanılan ortak değişkenler başlangıçta `false`'dir.
- Burada `key` karşılıklı dışlama, `waiting[]` dizisi ise sınırlı bekleme için dairesel sırasal erişim hakkı için kullanılmıştır.

```
boolean waiting[n];
boolean lock;

boolean test_and_set(boolean *target) {
    boolean rv = *target;
    *target = true;
    return rv;
}
```

```
do { /*Pi süreci*/
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;

    /* kritik bölüm */

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = false;
    else
        waiting[j] = false;

    /* sonraki bölüm */
} while (true);
```

# Mutex Kilitleri

- Önceki bölümde sunulan donanım tabanlı kritik bölüm problemi çözümleri hem karmaşık hem de uygulama geliştiriciler tarafından erişilemez yaklaşımlardır.
- Bunun yerine işletim sistemleri bu sorunun çözümü için yazılım tabanlı yaklaşımlar geliştirmişlerdir.
- Bu araçların en basit olanı **mutex** (**m****u****t****u****a****l** **e****x****c****l****u****s****i****o****n**) yaklaşımıdır
- Kritik bölüm probleminden kaçınmak ve race condition durumunu önlemek için mutex kilidi kullanılır.
- Bir süreç kritik bölüme girmeden önce kilidi elde eder (**acquire()**) ve kritik bölümü tamamladıktan sonra kilit serbest (**release()**) bırakılır.
- Burada kullanılan kilit fonksiyonları önceki bölümde sunulan donanımsal mekanizmalardan birini kullanırlar. Yani **acquire()** ve **release()** fonksiyonları atomic çalışırlar.



# Mutex Kilitleri

- Aşağıda örnek kod yapısı sunulan bu teknikteki en önemli sorun önceki yapılarda da görülebileceği üzere sürekli bir döngüde kilit kontrolü yapılmasıdır.
- Bir süreç kritik bölümü çalıştırırken diğer tüm süreçler sürekli döngü (**spinlock**) ile bekletilirler.
- Bu durum CPU'nun ortak kullanılan diğer süreçler için meşgul edilmesine neden olur.
- Ancak çok işlemcili sistemlerde spinlock, kilit bekleyen süreçler arasında context-switch gerektirmediği için kısa süreli kritik bölümler için avantaj sağlar. Özellikle thread'ler için performans sağlar.

```
acquire() {  
    while (!available);  
    available = false;  
}
```

```
release() {  
    available = true;  
}
```

```
do {  
    acquire();  
    /*kritik bölüm*/  
    release();  
    /*sonraki bölüm*/  
} while (true);
```

# Semaforlar

- Basit mutex mekanizmalarına alternatif olarak S olarak isimlendirilen semafor tamsayısı kullanılır.
- İki standart atomic operasyon tanımlanmıştır. `wait()` ve `signal()`

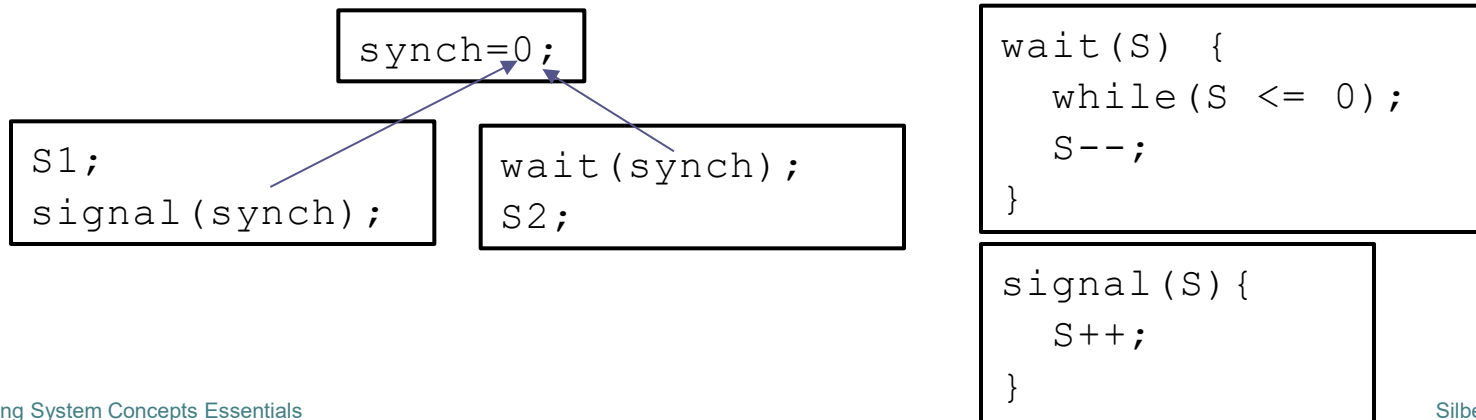
```
wait(S) {  
    while(S <= 0);  
    S--;  
}
```

```
signal(S) {  
    S++;  
}
```

- Bir süreç semafor tamsayısını değiştirdiğinde başka hiçbir süreç bu işleme erişemez.
- İşletim sistemleri **counting (sayma)** ve **binary (ikili)** semafor olarak isimlendirilen yapılar kullanır.

# Semaforlar

- Sayma semaforları sınırsız aralıkta çalışırken ikili semaforlar mutex kilitlerine benzer şekilde 0/1 değerleri alabilir.
- Sayma semaforları başlangıçta sınırlı sayıda kaynağı yönetmek için kaynak adetine kurulur.
- Bir süreç bir kaynağa erişmek istediğinde bir `wait()` işlemi başlatır. Kaynağı serbest bıraktığında `signal()` işlemini çağırır.
- `s` değeri 0 olduğunda tüm kaynaklar dolu anlamına gelir.
- `s` değeri 0'dan büyük olana kadar diğer süreçler engellenir.
- Örneğin aynı anda çalışan P1 ve P2 süreçleri için; P1'de tanımlı S1 işlemi P2'de tanımlı S2 işleminden önce yürütülmek zorunda olsa idi;



# Semafor Uygulamaları

- Mutex kilitlerinin CPU'yu meşgulde bekletmesi sorunundan dolayı `wait()` ve `signal()` semaforları **değiştirilmiştir**.
- Bir süreç, `wait()` operasyonunda tamsayı değerin pozitif olmadığını bulduğunda bekleme durumuna geçer. Böylece süreç, `block()` talimatıyla askıya alınarak bekleme listesine eklenir. Süreç tamamlandığında diğer süreçlerden biri `wakeup()` talimatıyla uyandırılır.

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        Bu süreci S->list'e ekle;  
        block();  
    }  
}
```

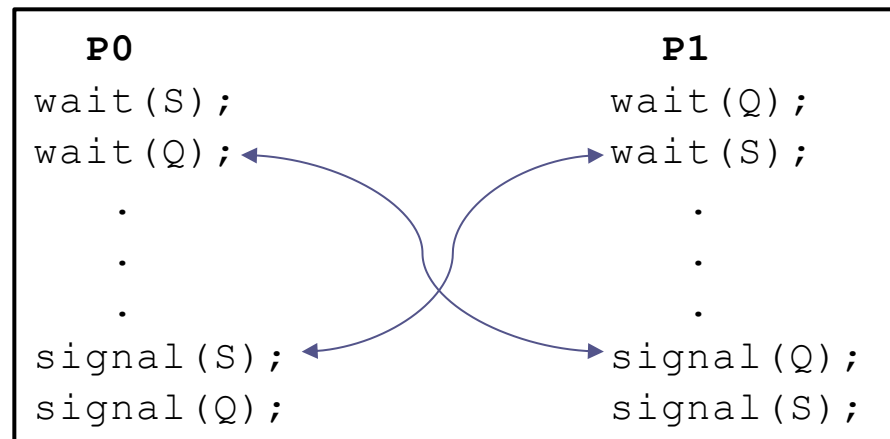
```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        P süreci S->list'den çıkart;  
        wakeup(P);  
    }  
}
```

# Semafor Uygulamaları

- Bir semafor'un negatif değeri, o semaforda bekleyen süreç sayısını ifade eder. Bekleyen süreçlerin listesi, her sürecin **PCB'si** ile ilişkilendirilerek tanımlanabilir. Böylece her semafor bir tamsayı değer ve bir PCB bağlantısı içerir.
- **Bounded-waiting** gereksinimini sağlamak için FIFO yapısında bir kuyruk tanımlanabilir.
- Bir semafor işleminin (wait/signal) atomic olması genel kuraldır. Bu durum tek işlemcili yapılarda **kesmelerin** engellemesi ile sağlanır.
- Ancak çok işlemcili yapılarda kesmeleri devre dışı bırakmak ciddi performans kayıplarına neden olabileceği için **compare\_and\_swap()** veya **spinlocks** gibi alternatif teknikler kullanılabilir.
- Dolayısıyla tam anlamıyla sistemin meşgul durumunun engellendiği söylenemez. Aksine uygulama programlarının giriş bölümünde meşgulde beklemesi yerine kritik bölümde bekletilmesi sağlanmıştır.
- Ancak kritik bölümler, genelde kısa oldukları için süreçlerin meşgulde bekleme oranları en aza indirilmiş olur. Ancak tersi de mümkündür.

# Semafor - Kilitlenme(deadlock)

- Bir semafor uygulamasında, bekleme kuyruktaki iki veya daha fazla süreç, kuyruktaki süreçlerden birinden kaynaklanan bir olaydan dolayı sonsuza kadar bekleme durumunda kalabilirler. Bu durum **deadlock** olarak isimlendirilir.
- Aşağıdaki örnekte  $P_0$  ve  $P_1$  süreçlerinin, S ve Q olarak isimlendirilen iki semafora eriştikleri varsayılmaktadır.



- $P_0$  wait(S),  $P_1$  wait(Q) çalıştırır. Ancak sonraki adımlar için;
- $P_0$ 'ın wait(Q)'yu çalıştırabilmesi için  $P_1$  signal(Q)'yu çalıştırması,  $P_1$ 'in wait(S)'yi çalıştırabilmesi için  $P_0$  signal(S)'yi çalıştırması gerekir

# Semafor - Öncelik çakışması

- Semafor planlama zorluklarından biri de öncelik çakışmasıdır (**priority inversion**).
- Çekirdek veri yapıları sıradan kilit yapıları ile korunduğu için, higher-priority bir süreç, kaynağı serbest bırakıncaya kadar lower-priority bir süreci beklemek zorundadır.
- Örneğin öncelik sıralaması  $P_L < P_M < P_H$  şeklinde 3 süreç düşünelim;
  - $P_L$  süreci kritik bölümünü çalıştırsın.  $P_H$  ile paylaştığı X kaynağına erişsin. Yani acquired(X)
  - $P_H$  süreci çalıştırılmak istendiğinde yüksek öncelikli olduğu için CPU context-switching ile süreci yürütecektir. Ancak  $P_H$  kritik bölümüne geldiğinde X kaynağı kilitli olduğu için  $P_L$ 'yi beklemeye geçecektir.  $P_H \rightarrow \text{WAITING}$ ,  $P_L \rightarrow \text{RUNNING}$
  - $P_M$  süreci yüklensin. Ancak kritik bölüme ihtiyacı olmadığı için ve  $P_L$ 'den daha yüksek önceliğe sahip olduğu için çalışacaktır. CPU context-switching ile  $P_L \rightarrow \text{READY}$ ,  $P_M \rightarrow \text{RUNNING}$
  - Bu durumda  $P_H$  kritik bölümden dolayı  $P_L$ 'nin,  $P_L$ 'de öncelikten dolayı  $P_M$ 'nin bitmesini bekler.
  - Dolayısıyla  $P_H$  yüksek önceliğe sahipken  $P_M$ 'den sonra yürütülür.
- İki den fazla öncelik seviyesine sahip sistemlerde bu sorunun çözümünde **priority-inheritance protocol** kullanılır.
- Bu protokole göre bir kaynak düşük öncelikli bir süreç tarafından tutuluyorken yüksek öncelikli bekleme listesindeki süreçlerin önceliğini geçici olarak miras alır.
- Yukarıdaki örnek için geçici olarak  $P_L$ , H önceliğine sahip koşar.

# Klasik Senkronizasyon Problemleri

Önerilen her yeni **senkronizasyon algoritmasını** test etmek için bazı klasik senkronizasyon problemleri kullanılır.

**1- Bounded-Buffer Problem:** Bu problem klasik üretici/tüketici problemi için paylaşılan tampon sorununun testi için kullanılır. Ortak veri yapılarında, mutex 1 ile başlatılır ve karşılıklı dışlama (**mutual exclusion**) sağlanır. **empty** ve **full** semaforları boş ve dolu tamponu tanımlar. Örnek kod yapısında üretici tüketici için **full** tamponunu, tüketici üretici için **empty** tamponunu kullanır.

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        block();
    }
}
```

```
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        wakeup(P);
    }
}
```

```
int n;
semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0
```

```
do { /*üretici*/
    . . .
    /* yeni bir ürün üret */
    . . .
    wait(empty);
    wait(mutex);
    . . .
    /*yeni ürünü tampona ekle*/
    . . .
    signal(mutex);
    signal(full);
} while (true);
```

```
do { /*tüketici*/
    wait(full);
    wait(mutex);
    . . .
    /*tampondan bir eleman sil*/
    . . .
    signal(mutex);
    signal(empty);
    . . .
    /* ürünü işle*/
    . . .
} while (true);
```



# Klasik Senkronizasyon Problemleri

**2- Readers-Writers Problemi:** Bir veritabanının birden fazla eşzamanlı süreç arasında paylaşıldığını varsayalım. Bazı süreçler veritabanını okumak isterken bazıları yazmak isteyebilir. Bu durum senkronizasyon problemine yol açar. Bu problemde eğer hiçbir **writer** nesneye erişim izni almamışsa **reader**'lar bekletilmez. Ancak bir **writer** nesneye yazmaya hazırsa hiçbir **reader** okumaya başlayamaz.

**mutex** semaforu **read\_count** güncellendiğinde karşılıklı dışlama için, **read\_count** değişkeni ise nesneyi okuyan süreç sayısında kullanılır. **rw\_mutex** semaforu **writers**'lar için karşılıklı dışlamayı sağlarken kritik bölüme giren ve çıkan ilk ve son **reader**'ları göstermek için kullanılır. Eğer bir **writer** kritik bölümde ise tüm **reader**'lar bekletilir.

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        block();
    }
}
```

```
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        wakeup(P);
    }
}
```

```
semaphore rw_mutex = 1;
semaphore mutex = 1;
int read_count = 0;
```

```
do { /*writers*/
    wait(rw_mutex);
    . . .
    /* yaz */
    . . .
    signal(rw_mutex);
} while (true);
```

```
do { /*readers*/
    wait(mutex);
    read_count++;
    if (read_count==1)
        wait(rw_mutex);
    signal(mutex);
    . . .
    /* oku */
    . . .
    wait(mutex);
    read_count--;
    if (read_count==0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```

# Klasik Senkronizasyon Problemleri

**3- Dining-Philosophers Problemi:** Düşünen ve yiyen 5 filozof olsun. Filozoflar 5 sandalyeyle çevrili dairesel bir masayı paylaşsınlar. Masanın merkezinde bir kase pirinç ve 5 tane yemek çubuğu olsun.

Bir filozof düşünürken diğerleriyle etkileşime girmez. Fakat filozof acıkır ve ona komşu olan (sağ-sol) **iki** yemek çubuğunu almaya çalışır. Ancak komşusunun elindeki çubuğu alamaz. Doyan filozof çubuğunu bırakır ve düşünmeye devam eder.

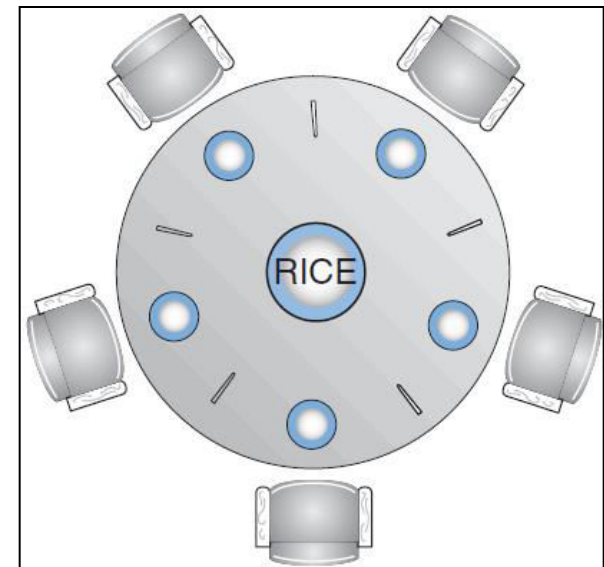
**chopstick** dizisinin tüm elemanları başlangıçta 1'e kurulur. Tüm filozoflar aynı anda yemek isterse tümü kendi sağındaki semaforu 0 edeceği için yine bir kilitlenme sorunu ortaya çıkacaktır. Bunun çözümü deadlocks konusu altında ele alınmıştır.

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        block();
    }
}
```

```
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        wakeup(P);
    }
}
```

```
semaphore chopstick[5];

do {
    wait(chopstick[i]);
    wait(chopstick[(i+1)%5]);
    . . .
    /* ye */
    . . .
    signal(chopstick[i]);
    signal(chopstick[(i+1)%5]);
    . . .
    /* düşün */
    . . .
} while (true);
```



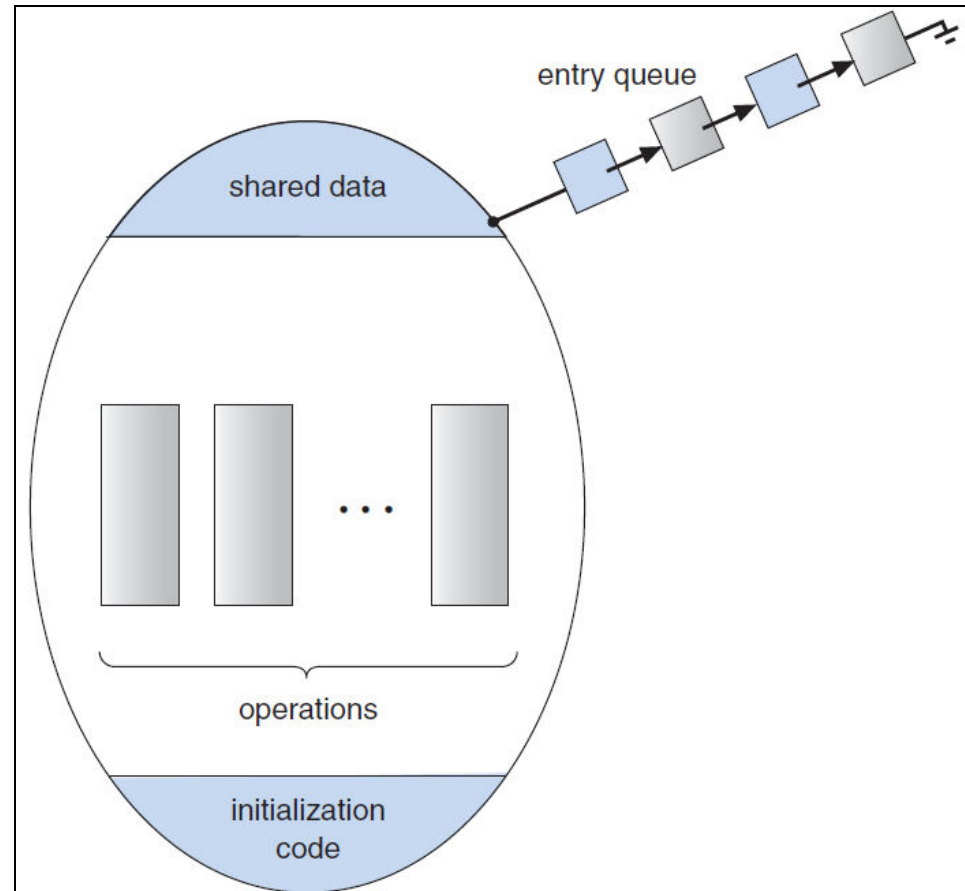
# Monitörler

- Senkronizasyon problemlerinde Semafor'lara alternatif olarak Monitörler kullanılır.
- Semaforların program geliştiriciler tarafından **wait()** ve **signal()** işlemleri yapmaları gereksinimi sorunlara neden olabilmektedir.
- Kritik bölüm geliştirmesinde senkronizasyonda ortaya çıkan sorunlar **deadlock** ve hatalı sonuçların oluşmasına neden olmaktadır.
- Bu tür hatalardan dolayı yüksek seviyeli dil yapılarına uygun yüksek seviyeli senkronizasyon yapısı olarak **monitörler** geliştirilmiştir.
- Monitörler **ADT**(abstract data type) tipindedir. ADT'ler, uygulama bağımsız veri tipi ve operasyonu sunarlar.
- Monitörler diğer süreçlerce doğrudan kullanılamaz.
- Monitör içerisinde tanımlı fonksiyon sadece yerel değişkenlere erişebilir.
- Monitörler, aynı anda monitör içinde sadece bir işlemin aktif olmasını sağlar.
- Sonuç olarak programcının senkronizasyon için ilave kısıt kodları yazmasına gerek kalmaz. C# ve Java dilleri monitör mekanizmasını kullanır.

```
monitor monitor_name {
    /*paylaşılan veri tanımı*/
    function P1 (...) {
        . . .
    }
    function P2 (...) {
        . . .
    }
    . . .
    function Pn (...) {
        . . .
    }
    initialization_code (...) {
        . . .
    }
}
```

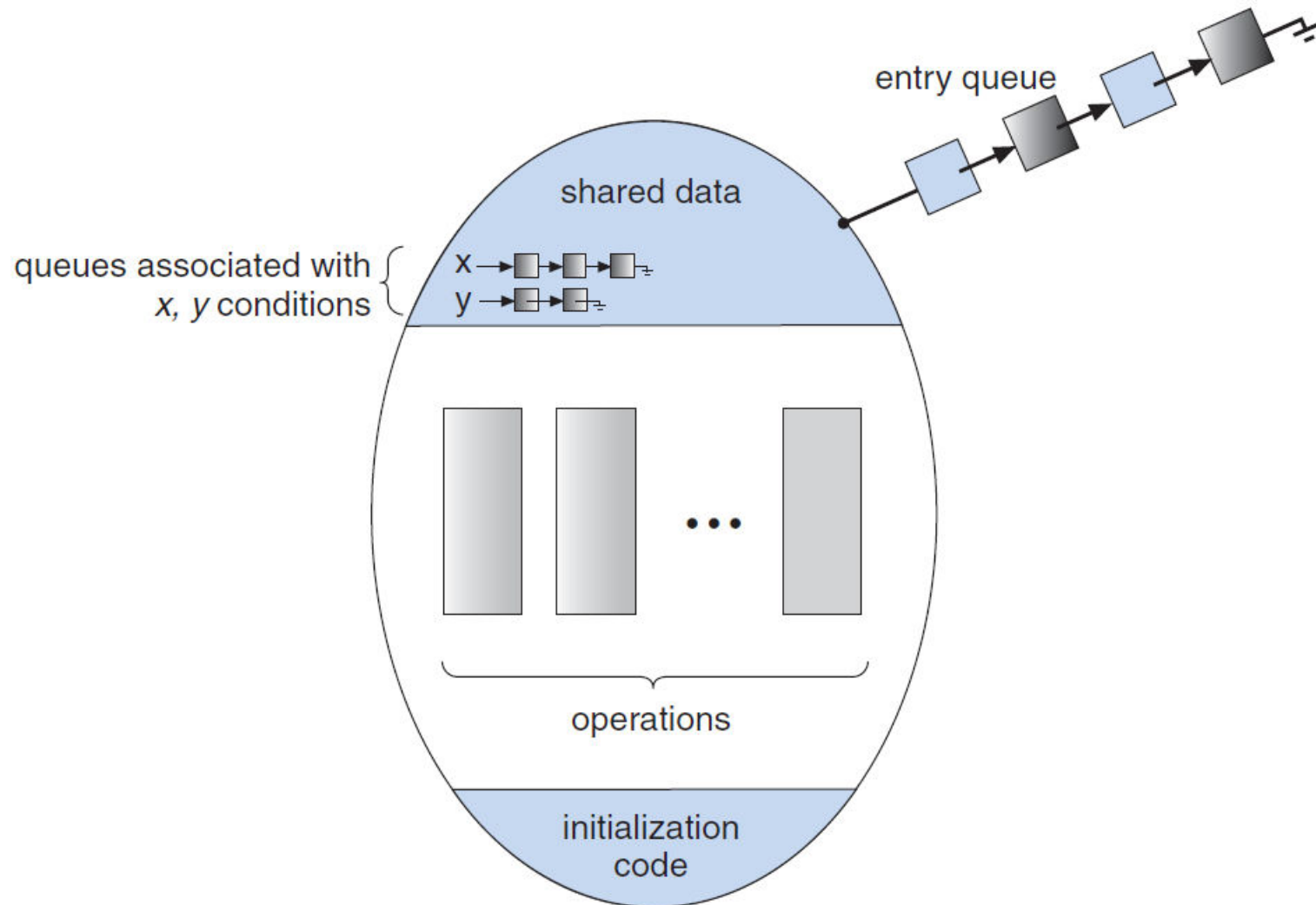
# Monitörler

- Ancak tanımlanmış monitör yapıları, bazı senkronizasyon şemalarını modellemek için yeterince güçlü değildir. Bu nedenle ilave senkronizasyon mekanizmaları tanımlamak gerekir. Bu mekanizmalar **condition** yapısı ile sağlanır.
- `condition x, y`; tanımı ile `x` ve `y` durumlarına bağlı çalışan `wait()` ve `signal()` kullanılabilir.
- `x.wait()`, ile `x` durumuna bağlı bir süreç bekletilebilir,
- `x.signal()` ile bir süreç çalışmasını sürdürebilir.
- `x` ile ilişkili `P` ve `Q` süreçleri olsun,
- Burada `P` süreci `x.signal()` çağırırsa,
- `Q` süreci beklemede ise çalışmaya başlayacak ve `P` sürecinin `Q` sürecini beklemesi gerekecektir.



# Monitörler

- x ve y durumlarına bağlı süreçlerin monitörde işleme alınması aşağıda sunulmuştur.



# Monitörler

**Dining-Philosophers Probleminin Çözümü:** Bu çözüm, her filozofun yalnızca iki yemek çubuğu alabilir olabilmesi kısıtını zorlar. Çözüm için filozofun üç durumunun tanımlandığı veri yapısı kullanılır.

```
enum {THINKING, HUNGRY, EATING} state[5];
```

- i. filozof durumunu `state[i]=EATING` yapabilmesi için onun iki komşusunun durumu EATING'den farklı olmalı;

```
(state[(i+4)%5] != EATING) ve (state[(i+1)%5] != EATING)
```

- Ayrıca filozof HUNGRY iken, yemek çubuklarını alamadığı durumda kendi durumunu beklemeye almak için; `condition self[5];` veri yapısı kullanılır.
- Bu problemin çözümü için her filozof yemeğe başlamadan önce `pickup(i)` operasyonunu çağırmak zorundadır. Bu operasyon başarıyla tamamlanırsa filozof yemek yiyebilir.
- Yemeğini bitiren filozof `putdown()` operasyonu ile sürece devam eder.

```
DiningPhilosophers.pickup(i);
...
yemek yer
...
DiningPhilosophers.putdown(i);
```

# Monitörler

## Dining-Philosophers Probleminin Çözümü:

```
monitor DiningPhilosophers
{
    enum {THINKING, HUNGRY, EATING} state[5];
    condition self[5];

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait();
    }

    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}
```

```
.....
void test(int i) {
    if ((state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING)) {
        state[i] = EATING;
        self[i].signal();
    }
}

initialization code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
}
```

```
DiningPhilosophers.pickup(i);
    yemek yer
DiningPhilosophers.putdown(i);
```

# Monitörler

**Monitör Semafor Entegrasyonu:** Semafor kullanan her monitör uygulaması için başlangıç değeri `mutex=1` olan bir semafor sağlanabilir.

- Bir süreç monitöre girmeden önce bekleme (`wait(mutex)`), monitörden ayrıldıktan sonra serbestlik (`signal(mutex)`) kullanılır.
- Bir süreç `signal`'den önce, yürüyen sürecin beklemeye geçmesi yada monitörden ayrılmasını beklemelidir.
- Bir tamsayı değişken olan `next_count` ile askıda/sırada olan süreçlerin sayısı takip edilir.
- Her `x` durumu için `x_sem` ve `x_count` semaforları ile yalnızca bir sürecin monitör içerisinde aktif olması sağlanır.

```
wait(mutex);
...
fonksiyonlar
...
if(next_count > 0)
    signal(next);
else
    signal(mutex);
```

```
WAIT:
x_count++;
if(next_count > 0)
    signal(next);
else
    signal(mutex);
wait(x_sem);
x_count--;
```

```
SIGNAL:
if(x_count > 0) {
    next_count++;
    signal(x_sem);
    wait(next);
    next_count--;
}
```



# Monitörler

**Monitör ile Süreçlerin sürdürümü:** Bir monitör içinde aynı durum için bekleyen bir çok süreç olduğunda monitörün `signal(next)` işlemi hangi süreci devreye almalıdır?

- Bu sorun genellikle **first-come, first-served (FCFS)** yaklaşımı ile çözümlense de bu bazen yetersiz olabilmektedir.
- Bu durumda **conditional-wait** yapısı kullanılır. `x.wait(c)` ; gibi.
- Burada `c` bekleme işlemi çağrıldığında sürecin değerlendirildiği öncelik (priority) numarasıdır ve küçük olan değer işleme alınır.
- Ancak monitörlerde de semaforlarda olduğu gibi erişim sırası sorunu yaşanacaktır. Bu sorunlar (1) Bir süreç kaynağı ilk kullanım hakkı olmaksızın kaynağa erişebilir, (2) Bir süreç kaynağa erişim hakkını aldıktan sonra serbest bırakmayabilir, (3) Bir süreç istenilmediği halde kaynağı serbest bırakmayı deneyebilir, (4) Bir süreç aynı kaynağa iki kez erişmek isteyebilir.
- Dolayısıyla semafor kullanımında ifade edilen programlama noktasındaki çekincelere benzer sorunlar monitör kullanımında da vardır. Fakat bu zorluklar artık derleyiciler noktasında oluşmaktadır.

# Monitörler

**Monitör ile Süreçlerin sürdürümü:** Aynı kaynağa erişmek isteyen süreçler için bu kaynağın tahsisini gösteren ResourceAllocator monitörü tanımlanmıştır. Bu kaynağın tahsisini isteyen her süreç kaynağı ortalama ne kadar süre kullanacağını planlar. Monitör en kısa süre kaynağı kullanmayı talep eden sürece ayırır. Ancak semaforlarda olduğu gibi süreç aşağıdaki sırasallığa uymak zorundadır.

```
R.acquire(t);  
...  
kaynağa erişim;  
...  
R.release();
```

```
monitor ResourceAllocator {  
    boolean busy;  
    condition x;  
    void acquire(int time) {  
        if (busy)  
            x.wait(time);  
        busy = true;  
    }  
  
    void release() {  
        busy = false;  
        x.signal();  
    }  
  
    initialization code() {  
        busy = false;  
    }  
}
```

# Monitörler

**Java Monitörleri:** Java, thread'lerin eşzamanlılığı için monitör mekanizmasına benzer bir yapı sunar. Her java nesnesi tekil bir kilit ile ilişkilidir. `synchronized` bir metod tanımlandığında metodu çağıran nesne için kilitin sahibi olmayı gerektirir.

```
public class SimpleClass {  
    . . .  
    public synchronized void safeMethod(){  
        . . .  
        /* safeMethod() içeriği */  
        . . .  
    }  
}
```

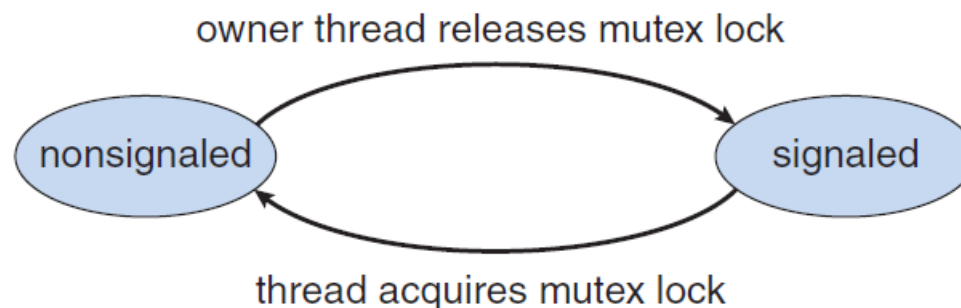
```
SimpleClass sc = new SimpleClass();
```

- `sc.safeMethod()` çağrıldığında `sc` nesnesi kilitin sahibi olur. Eğer kilit başka bir thread'in ise kilit serbest kalıncaya kadar ilgili nesne bekletilir. Thread tamamlandığında kilit serbest bırakılır.
- Java monitörleri `wait()` ve `signal()` metodlarına benzer `wait()` ve `notify()` metodları sağlar. `java.util.concurrent` paketi mutex kilitleri, durum değişkenleri ve semaforlar için destek sağlar.

# Senkronizasyon Örnekleri

**Windows:** Windows işletim sistemi çok işlemcili ve gerçek zamanlı uygulamalar için destek sağlayan bir multithread çekirdeğe sahiptir.

- Windows tek işlemcili yapılarda global kaynaklara erişim için tüm kesme isteklerini geçici olarak engeller. Çok işlemcili yapılarda ise spinlock kullanılır.
- Çekirdek dışında çalışan thread'ler için **dispatcher** nesneler sağlanır. Bu nesneler ile iş parçacıkları, mutex kilitleri ve semaforlar gibi senkronizasyon mekanizmaları ile korunurlar.
- Şekilde görüldüğü üzere bir **signaled** durumunda olan bir nesneye erişmek isteyen thread engellenmez. Oysa nesne **nonsignaled** durumunda ise thread engellenecektir.
- Bir thread nonsignaled dispatcher nesne tarafından bloklandığında thread'in durumu ready'den waiting'e geçecektir ve nesnenin waiting object kuyruğuna eklenecektir.



# Senkronizasyon Örnekleri

**Linux:** Linux çekirdeği senkronizasyon için farklı mekanizmalar sunmaktadır. Bunlardan en basit olanı atomic yapılar üzerinden kesintisiz işlemler sağlanır.

```
atomic_t counter;
int value;
atomic_set(&counter, 5);      /* counter = 5 */
atomic_add(10, &counter);     /* counter = counter + 10 */
atomic_sub(4, &counter);      /* counter = counter - 4 */
atomic_inc(&counter);          /* counter = counter + 1 */
value = atomic_read(&counter); /* value = 12 */
```

Ayrıca çekirdek içerisinde kritik bölümleri korumak için mutex kilitleri mevcuttur. (mutex\_lock(), mutex\_unlock())

Ayrıca linux çekirdeği spinlock ve semaforlarda sağlar. Ancak tek işlemcili (gömülü sistemler gibi) yapılarda linux çekirdeği spinlock yerine nonpreemption (kesilemeyen zamanlama) davranır.

| single processor           | multiple processors |
|----------------------------|---------------------|
| Disable kernel preemption. | Acquire spin lock.  |
| Enable kernel preemption.  | Release spin lock.  |

# Alternatif Yaklaşımlar

**Transactional Memory:** Bu yaklaşım temelde veritabanı teorisine dayanmaktadır. Burada bellek okuma/yazma operasyonu sırasal olarak atomic şekilde yürütülür. Eğer tüm işlemler tamamlanırsa bellek aktarılır. Eğer bir sorun oluşursa **rolled-back** yapılır.

**OpenMP:** C, C++ ve Fortran derleyicileri için derleme direktifleri sunan OpenMP API'yı shared-memory nesneleri üzerinde eşzamanlı çalışmayı sağlayan `#pragma omp critical` gibi kritik bölüm tanımlama talimatı sunar. Böylece ilgili bölüm aynı anda sadece bir thread tarafından işlenebilir.

**Fonksiyonel programlama dilleri:** C, C++, Java ve C# gibi programlama dilleri prosedurel (imperative) dillerdir. Yani bu diller durum tabanlı dillerdir ve program durumu değişebilir. Erlang ve Scala gibi fonksiyonel dillerin temel farkı ise durumlarının korunuyor olmasıdır. Fonksiyonel dillerde durum değişimi sözkonusu olmadığı için yarış durumu ve deadlocks sorunları da oluşmaz.

# Deadlocks / Kilitlenmeler

- Multiprogram ortamlarında birkaç süreç sınırlı kaynakları kullanmak için rekabet içerisindedir.
- Bir süreç bir kaynağa erişmek istediğinde eğer o kaynak uygun değilse süreç bekleme durumuna girer.
- Bazen bekleme durumundaki süreç diğer bekleme durumundaki süreçler tarafından isteği engellendiği için asla durumunu değiştiremez. Bu durum **deadlock** olarak isimlendirilir.
- Deadlock durumu mantıksal kaynaklarda (dosya, semafor, monitör) olabileceği gibi fiziksel kaynaklarda (printer, driver, memory, cpu) da olabilir.
- Örneğin 3 süreç sistemdeki 3 CD-RW kullanıyorken her bir süreç diğer CD-RW'lardan birine erişmek istediğinde deadlock oluşur. Bu duruma aynı kaynak türlü deadlock denir.
- Örneğin bir DVD ve yazıcısı olan bir sistemde  $P_i$  yazıcıyı,  $P_j$  DVD'yi kullanıyor olsun. Eğer  $P_i$  süreci DVD'yi  $P_j$ 'de yazıcıyı talep ederse deadlock oluşur ve buna da farklı kaynak türlü deadlock denir.
- Multithreading yapıdaki uygulamalar açısından kilitlenme/deadlock durumu özellikle dikkat edilmesi gereken bir sorundur.

# Deadlocks / Kilitlenmeler

- Bir kilitlenme durumunun oluşmasına genellikle aşağıdaki 4 durum yol açar;
  1. **Mutual exclusion:** Bir kaynak bir seferde yalnızca bir süreç tarafından kullanılabilir. Başka bir süreç bu kaynağa erişmek isterse kaynak serbest bırakılıncaya kadar bekletilmelidir.
  2. **Hold – Wait:** Bir süreç, diğer süreçler tarafından tutulan kaynaklara erişim isteği için bekletilmeli ve en az bir kaynağa erişebilmeli.
  3. **No preemption:** Bir süreç bir kaynağı kendi isteğiyle serbest bırakmadığı sürece kesilmemeli.
  4. **Circular wait:**  $P_0, P_1 \dots P_n$  şeklinde bir süreç dizesi, birbirinin kullandığı kaynakları dairesel olarak beklememeli.
- Deadlock durumu **resource-allocation graph** olarak isimlendirilen bir graf modeli ile tanımlanması kolaydır.  $P = \{P_0, P_1 \dots P_n\}$  sistemdeki tüm etkin süreçleri,  $R = \{R_0, R_1 \dots R_n\}$  sistemdeki tüm kaynakları tanımlasın. E olarak isimlendirilen kenarlık setinde ise  $P_i \rightarrow R_j$  bir kaynak isteğini (**request edge**),  $R_j \rightarrow P_i$  bir kaynak tahsisini (**assignment edge**) ifade eder.
- Bir kaynağın birden fazla örneği (instance) olabilir. Dolayısıyla bir kaynak birden fazla sürece her bir örneği bir sürece tahsis edilmek üzere ayrılabilir.



# Deadlocks / Kilitlenmeler

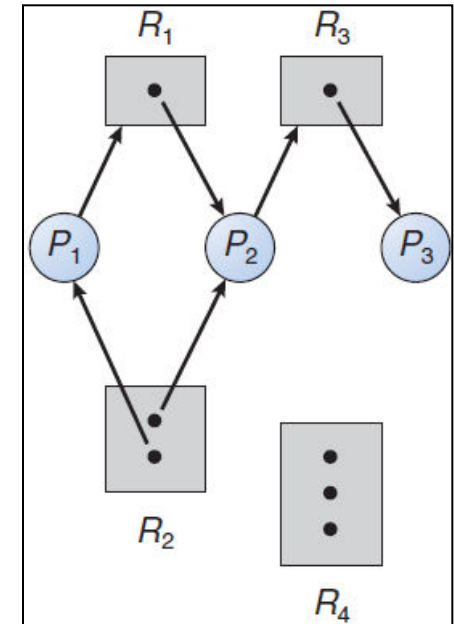
Aşağıdaki resource-allocation graph'ı için;

- P, R ve E dizileri;

$P = \{P_1, P_2, P_3\}$  süreçleri ve  $R = \{R_1, R_2, R_3, R_4\}$  kaynakları,  
 $E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$

- Kaynak instance'ları;  $R_1$  ve  $R_3$  bir instance,  $R_2$  iki instance ve  $R_4$  üç instance
- Süreç durumları;
  - $P_1$  süreci  $R_2$  kaynağının bir instance'ını tutuyor ve  $R_1$  kaynağı için beklemede,
  - $P_2$  süreci  $R_1$  ve  $R_2$ 'nin bir instance'ını tutuyor  $R_3$  kaynağı için beklemede,
  - $P_3$  süreci  $R_3$  kaynağını tutuyor.

Eğer E dizisi bir dairesel çevrim içerseydi sistemde deadlock olduğu anlamına gelirdi. Ancak bir kaynağın boşta bir instance'ı olursa, bir dairesel çevrim oluşsa dahi deadlock olduğu anlamına gelmez.



# Deadlocks / Kilitlenmeler

Aynı örnekte  $P_3$  süreci  $R_2$  kaynağını talep etseydi deadlock durumu oluşurdu.

$$P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$

$$P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$$

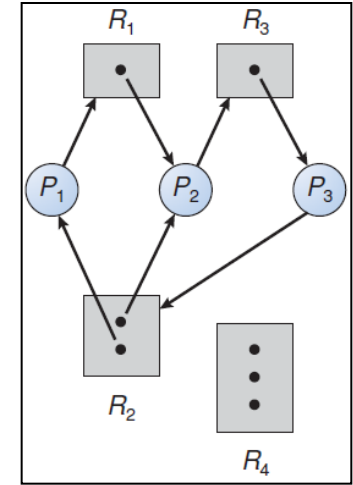
Bu durumda  $P_1$ ,  $P_2$  ve  $P_3$  süreçleri deadlock durumundadır.

$P_2$  süreci  $R_3$  kaynağını beklemektedir ve  $R_3$ 'ü  $P_3$  tutmaktadır,

$P_3$  süreci  $R_2$  kaynağını beklemektedir ve  $R_2$ 'yi  $P_2$  tutmaktadır,

$P_3$  süreci  $P_1$  veya  $P_2$ 'nin  $R_2$ 'yi serbest bırakmasını beklemektedir ancak

$P_1$  aynı zamanda  $P_2$ 'nin  $R_1$ 'i serbest bırakmasını beklemektedir.

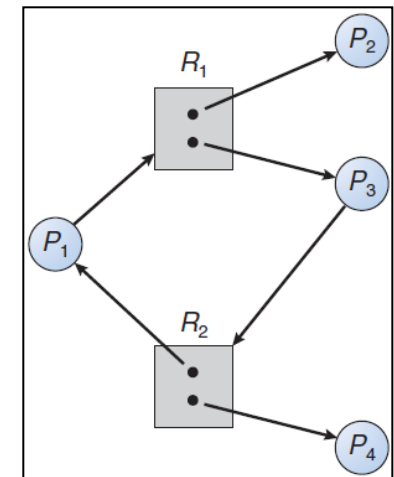


Ancak deadlock durumu yandaki grafa oluşmaz.

$$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$$

gibi bir dairesel çevrim olsada;

Çünkü  $P_4$  süreci  $R_2$  kaynağının bir instance'ını serbest bıraktığında  $P_3$  süreci dairesel çevrimi kıracaktır.



# Deadlocks / Kilitlenmeler

- Ortaya konulan kilitlenme / deadlock sorunu çözmek için üç yoldan biri tercih edilebilir;
  1. Bir sistem kilitlenmelerin önüne geçebilmek ve sistemin asla kilitlenmemesi için bir protokol kullanabilir.
  2. Sistem deadlock durumuna düştüğünü anlayabilir ve sistemi bu durumdan kurtarabilecek çözümlere izin verir.
  3. Sistem tamamen deadlock durumunu görmezden gelir ve tüm süreçleri asla deadlock oluşmayacak şekilde sürdürmeye devam eder.
- Üçüncü yöntem UNIX ve Windows da dahil olmak üzere pek çok işletim sistemi tarafından tercih edilmektedir.
- Bu sistemlerde deadlock yönetimi uygulama geliştiricilere bırakılmıştır.
- Yukarıdaki üç yöntemden hangisi tercih edilirse edilsin mutlak anlamda deadlock durumunun önüne geçilebileceği söylenemez.
- Pekçok araştırmacı en iyi çözümün 3 yönetimin birlikte kullanımıyla sağlanabileceğini iddia etmektedir.